dev ops lab 6:compte-rendu

PARTIE 1 — Approche impérative (Shell Provisioner)

1) Exploration du Vagrantfile

Dans le dossier `lab/part-1`, j'ai consulté le fichier `Vagrantfile` fourni.

```
PS C:\lab\part-1> cd C:\lab\part-1
>> dir
>>


    Répertoire : C:\lab\part-1


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        13/02/2026     17:03              9 .gitignore
-a----        13/02/2026     17:03            863 Vagrantfile
```

2) Création de la VM

Commande exécutée :Vagrant up

```
>> C:\lab\part-1> vagrant up
Bringing machine 'centos_server' up with 'virtualbox' provider...
==> centos_server: Importing base box 'centos/7'...
==> centos_server: Matching MAC address for NAT networking...
==> centos_server: Checking if box 'centos/7' version '2004.01' is up to date...
==> centos_server: Setting the name of the VM: centos.server.local
==> centos_server: Clearing any previously set network interfaces...
==> centos_server: Preparing network interfaces based on configuration...
    centos_server: Adapter 1: nat
==> centos_server: Forwarding ports...
    centos_server: 22 (guest) => 2222 (host) (adapter 1)
==> centos_server: Running 'pre-boot' VM customizations...
==> centos_server: Booting VM...
==> centos_server: Waiting for machine to boot. This may take a few minutes...
    centos_server: SSH address: 127.0.0.1:2222
    centos_server: SSH username: vagrant
    centos_server: SSH auth method: private key
    centos_server:
    centos_server: Vagrant insecure key detected. Vagrant will automatically replace
    centos_server: this with a newly generated keypair for better security.
    centos_server:
    centos_server: Inserting generated public key within guest...
    centos_server: Removing insecure key from the guest if it's present...
    centos_server: Key inserted! Disconnecting and reconnecting using new SSH key...
==> centos_server: Machine booted and ready!
==> centos_server: Checking for guest additions in VM...
    centos_server: No guest additions were detected on the base box for this VM! Guest
    centos_server: additions are required for forwarded ports, shared folders, host only
    centos_server: networking, and more. If SSH fails on this machine, please install
    centos_server: the guest additions and repackage the box to continue.
    centos_server:
    centos_server: This is not an error message; everything may continue to work properly,
    centos_server: in which case you may ignore this message.
==> centos_server: Rsyncing folder: /cygdrive/c/lab/part-1/ => /vagrant
==> centos_server: Running provisioner: shell...
    centos_server: Running: inline script
    centos_server: Hello, World
PS C:\lab\part-1>
```

La VM CentOS s'est correctement créée et provisionnée. Le script shell intégré au Vagrantfile s'est exécuté, affichant : Hello,World.

3)Connexion à la VM

vagrant ssh ouvre une session dans la vm permettant de lancer les commandes bash.

```
PS C:\lab\part-1> vagrant ssh
>>
[vagrant@localhost ~]$ ls
[vagrant@localhost ~]$ pwd
/home/vagrant
[vagrant@localhost ~]$ cat/vagrant/Vagrant
-bash: cat/vagrant/Vagrantfile: No such fi
[vagrant@localhost ~]$ cat /vagrant/Vagran
# -*- mode: ruby -*-
# vi: set ft=ruby :

Vagrant.configure("2") do |config|

  # Do not pay attention to this parameter
  if Vagrant.has_plugin?("vagrant-vbguest"
    config.vm.provider :virtualbox do |vb|
      config.vbguest.auto_update = false
    end
  end

  # Define the VM
  config.vm.define "centos_server" do |ser
    # Specify the Vagrant box to use
    server.vm.box = "centos/7"
    # Specify the VM specs when using the
    server.vm.provider "virtualbox" do |vb
      vb.name =  "centos.server.local"
```

On a affiché le Vagrantfile depuis la VM.

4) modification du provisioning

Dans le fichier vagrantfile qu'on ouvre dans vscode on modifie la ligne suivante

```
# Start provisioning
config.vm.provision "shell",
  inline: "echo Hello, World"
```

Avant on avait:
inline:"echo Hello, World"
après la modification :
inline: "echo I am provising…"

```
PS C:\lab\part-1> C:\lab\part-1\Vagrantfile
>>
PS C:\lab\part-1> vagrant provision
>>
==> centos_server: Running provisioner: shell...
    centos_server: Running: inline script
PS C:\lab\part-1> vagrant ssh
Last login: Fri Feb 13 16:22:32 2026 from 10.0.2.2
[vagrant@localhost ~]$ cat /etc/hosts
127.0.0.1   localhost localhost.localdomain localhost4 localhost4.localdom
ain4
::1         localhost localhost.localdomain localhost6 localhost6.localdom
ain6
127.0.0.1  mydomain-1.local
[vagrant@localhost ~]$
```

```
>>
PS C:\lab\part-1> vagrant provision
>>
==> centos_server: Running provisioner: shell...
    centos_server: Running: inline script
    centos_server: I am provisioning...
PS C:\lab\part-1>
```

ainsi on a le fichier /etc/vagrant_provisioned_at qui a bien été créé et qui contient la date du provisioning.

```
>>
S C:\lab\part-1> vagrant provision
>>
==> centos_server: Running provisioner: shell...
    centos_server: Running: inline script
    centos_server:vagrant sshsioning...
> C:\lab\part-1>
ast login: Fri Feb 13 16:45:39 2026 from 10.0.2.2
vagrant@localhost ~]$ vagrant ssh
bash: vagrant: command not found
vagrant@localhost ~]$ cat /etc/vagrant_provisioned_at
ri Feb 13 16:57:34 UTC 2026
```

La VM a bien exécuté le nouveau script, et un fichier `/etc/vagrant_provisioned_at` a été créé, contenant la date du provisioning.

PARTIE 2 — Installation GitLab avec Vagrant + Ansible (déclaratif)
Objectif:Créer une VM Rocky Linux 8, puis laisser Ansible installer GitLab automatiquement via les playbooks fournis.

1) on prépare l'environnement virtuel

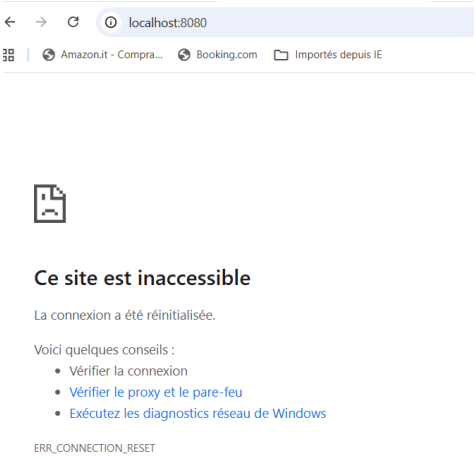
   Dans le dossier `lab/part-2`, j'ai utilisé le Vagrantfile fourni pour créer une VM

2) on creer une vm avec la commande vagrant up

La box `generic/rocky8` a été téléchargée puis provisionnée. Ansible (via `ansible_local`) a automatiquement installé GitLab et ses dépendances.

```
PS C:\lab\part-2> vagrant up
>>
Bringing machine 'gitlab_server' up with 'virtualbox' provider...
==> gitlab_server: Box 'generic/rocky8' could not be found. Attempting to find and install...
    gitlab_server: Box Provider: virtualbox
    gitlab_server: Box Version: >= 0
==> gitlab_server: Loading metadata for box 'generic/rocky8'
    gitlab_server: URL: https://vagrantcloud.com/api/v2/vagrant/generic/rocky8
==> gitlab_server: Adding box 'generic/rocky8' (v4.3.12) for provider: virtualbox (amd64)
    gitlab_server: Downloading: https://vagrantcloud.com/generic/boxes/rocky8/versions/4.3.12/providers/
virtualbox/amd64/vagrant.box
    gitlab_server:
    gitlab_server: Calculating and comparing box checksum...
==> gitlab_server: Successfully added box 'generic/rocky8' (v4.3.12) for 'virtualbox (amd64)'!
==> gitlab_server: Importing base box 'generic/rocky8'...
==> gitlab_server: Matching MAC address for NAT networking...
==> gitlab_server: Checking if box 'generic/rocky8' version '4.3.12' is up to date...
==> gitlab_server: Setting the name of the VM: gitlab.server.local
==> gitlab_server: Fixed port collision for 22 => 2222. Now on port 2200.
Vagrant is currently configured to create VirtualBox synced folders with
the `SharedFoldersEnableSymlinksCreate` option enabled. If the Vagrant
guest is not trusted, you may want to disable this option. For more
information on this option, please refer to the VirtualBox manual:

  https://www.virtualbox.org/manual/ch04.html#sharedfolders

This option can be disabled globally with an environment variable:

  VAGRANT_DISABLE_VBOXSYMLINKCREATE=1

or on a per folder basis within the Vagrantfile:

  config.vm.synced_folder '/host/path', '/guest/path', SharedFoldersEnableSymlinksCreate: false
==> gitlab_server: Clearing any previously set network interfaces...
==> gitlab_server: Preparing network interfaces based on configuration...
    gitlab_server: Adapter 1: nat
    gitlab_server: Adapter 2: hostonly
==> gitlab_server: Forwarding ports...
    gitlab_server: 80 (guest) => 8080 (host) (adapter 1)
    gitlab_server: 22 (guest) => 2200 (host) (adapter 1)
==> gitlab_server: Running 'pre-boot' VM customizations...
==> gitlab_server: Booting VM...
```

3)On test l'installation en allant sur http://localhost:8080



En ouvrant http://localhost:8080 dans le navigateur, la page GitLab ne s'affiche pas encore. Cependant, GitLab répond bien en interne via les endpoints de santé (curl dans la VM). L'installation est donc fonctionnelle, même si l'interface web n'est pas encore accessible depuis l'hôte

4)mise à jour des playbooks

```
PS C:\lab\part-2> cd C:\lab\part-2
PS C:\lab\part-2> vagrant upload playbooks /vagrant/playbooks gitlab_server
Uploading playbooks to /vagrant/playbooks
```

```
s":502}
[vagrant@rocky8 ~]$ curl http://127.0.0.1:8080/-/liveness
{"error":"nginx returned 502 when communicating with gitlab-workhorse","statu
s":502}
[vagrant@rocky8 ~]$ vagrant upload playbooks /vagrant/playbooks gitlab_server
-bash: vagrant: command not found
[vagrant@rocky8 ~]$ vagrant provision
-bash: vagrant: command not found
[vagrant@rocky8 ~]$ exit
logout
PS C:\lab\part-2> cd C:\lab\part-2
PS C:\lab\part-2> vagrant upload playbooks /vagrant/playbooks gitlab_server
Uploading playbooks to /vagrant/playbooks
Upload has completed successfully!

  Source: playbooks
  Destination: /vagrant/playbooks
PS C:\lab\part-2> vagrant provision
==> gitlab_server: Running provisioner: ansible_local...
Vagrant gathered an unknown Ansible version:


and falls back on the compatibility mode '1.8'.

Alternatively, the compatibility mode can be specified in your Vagrantfile:
https://www.vagrantup.com/docs/provisioning/ansible_common.html#compatibility
_mode
    gitlab_server: Running ansible-playbook...

PLAY [gitlab_server] ***********************************************************
***
```

Les commandes Vagrant doivent être exécutées **sur la machine hôte**, pas dans la VM.

PARTIE 3 — Health Checks GitLab

3.1 — Test manuel du health check

```
PS C:\WINDOWS\system32> cd C:\lab\part-2
PS C:\lab\part-2> vagrant ssh
Last login: Fri Feb 13 20:52:55 2026 from 10.0.2.2
[vagrant@rocky8 ~]$ curl http://127.0.0.1:8080/-/health
nginx returned 502 when communicating with gitlab-workhorse
[vagrant@rocky8 ~]$
```

Le health check retourne un code 502, indiquant que GitLab n'est pas encore complètement opérationnel.

2)lire le fichier main.yml du role healthchecks

```
cat: /vagrant/lab/part-2/playbooks/roles/gitlab/healthchecks/tasks/main.yml:
No such file or directory
[vagrant@rocky8 ~]$ ls /vagrant
main.yml  playbooks  Vagrantfile
[vagrant@rocky8 ~]$ ls /vagrant/playbooks
roles  run.yml
[vagrant@rocky8 ~]$ ls /vagrant/playbooks/roles
gitlab
[vagrant@rocky8 ~]$ ls /vagrant/playbooks/roles/gitlab
healthchecks  install
[vagrant@rocky8 ~]$ ls /vagrant/playbooks/roles/gitlab/healthchecks
tasks
[vagrant@rocky8 ~]$ ls /vagrant/playbooks/roles/gitlab/healthchecks/tasks
main.yml
[vagrant@rocky8 ~]$ cat /vagrant/playbooks/roles/gitlab/healthchecks/tasks/ma
in.yml
---
# OPTIONAL TASKS (needed to run checks from the host machine)
# - name: Uncomment the GitLab IP whitelist line
#   replace:
#     path: /etc/gitlab/gitlab.rb
#     regexp: '^# (gitlab_rails\['monitoring_whitelist'\] = \[.*)\]$'
#     replace: '\1, ''20.20.20.1'']'
```

```
# - name: Uncomment the GitLab IP whitelist line
#   replace:
#     path: /etc/gitlab/gitlab.rb
#     regexp: '^# (gitlab_rails\['monitoring_whitelist'\] = \[.*)\]$'
#     replace: '\1, ''20.20.20.1'']'

# - name: Reconfigure GitLab
#   command: gitlab-ctl reconfigure

# - name: Restart unicorn
#   command: gitlab-ctl restart unicorn
#   retries: 2
# END OF OPTIONAL TASKS

- name: Check GitLab health
  uri:
    url: http://127.0.0.1:8080/-/health
    return_content: yes
  # Register the output of the module in a variable
  register: gitlab_health

- name: Print GitLab health
  debug:
    msg: "{{ gitlab_health.content }}"
```

le rôle ne contient qu'un health check ,il utilise le module uri et affiche le résultat avec debug

Ainsi les commandes renvoient les mêmes erreurs

```
# Run the 2 other kind of health checks below
[vagrant@rocky8 ~]$ curl http://127.0.0.1:8080/-/readiness
{"error":"nginx returned 502 when communicating with gitlab-workhorse","statu
s":502}
[vagrant@rocky8 ~]$ curl http://127.0.0.1:8080/-/readiness
{"error":"nginx returned 502 when communicating with gitlab-workhorse","statu
s":502}
[vagrant@rocky8 ~]$ curl http://127.0.0.1:8080/-/liveness
{"error":"nginx returned 502 when communicating with gitlab-workhorse","statu
s":502}
[vagrant@rocky8 ~]$
```

Le readiness check retourne également 502, ce qui confirme que GitLab n'est pas encore prêt à accepter des requêtes.Le liveness check retourne aussi 502, indiquant que GitLab n'est pas encore pleinement opérationnel